

AI Vehicle Enrichment Pipeline

Technical Architecture & Implementation Guide

OtoPair | AppFlow Studios

March 2026

Executive Summary

A multi-stage, AI-powered pipeline that transforms a 17-character VIN into a complete vehicle intelligence profile: fluid specs, OEM part numbers, maintenance intervals, service pricing, and mechanical attributes. Built on Convex + Claude Sonnet 4.5 with web search, costing ~\$0.02 per vehicle enrichment.

Table of Contents

1	Pipeline Overview & Architecture	3
2	Stage 1: VIN Input (User Entry Points)	4
3	Stage 2: NHTSA Decode + AI Normalization	5
4	Stage 3: AI Enrichment Overview	6
5	Call 1A: Fluids, Intervals & Vehicle Attributes	7
6	Call 1B: OEM Part Numbers (Year-Pinned)	8
7	Year-Pinning & Part Validation (Layer 3)	9
8	Gap Fill: Sibling Cross-Reference + AI Retry	10
9	Call 2: Service Pricing Intelligence	11
10	Data Schema & Storage Architecture	12
11	Confidence Scoring & Review System	13
12	Five Key Improvements Over Previous Code	14
13	Practical Flow Summary	15
14	Implementation Status & Next Steps	16

1. Pipeline Overview & Architecture

The Big Picture

The OtoPair AI Enrichment Pipeline converts a raw VIN into a full vehicle intelligence profile through 4 sequential stages. The pipeline is event-driven: adding a vehicle triggers background enrichment automatically via Convex scheduler.

Architecture Flow

```
VIN Input (user)
|-> Stage 2: NHTSA Decode + AI Normalization
|   |-> makes -> models -> trims -> engines -> chassis_variants
|-> Stage 3: AI Enrichment (4 Claude Calls)
|   |-> Call 1A: Fluids, Intervals, Vehicle Attributes -> engine_specs
|   |-> Call 1B: OEM Part Numbers (year-pinned) -> vehicle_specs, trim_specs
|   |-> Gap Fill: Sibling cross-ref + targeted AI retry
|   |-> Call 2: Service Pricing -> service_vehicle_specs
|   |-> Audit Trail -> ai_enrichment_logs
```

Tech Stack

- **AI Model:** Claude Sonnet 4.5 (claude-sonnet-4-5-20250929)
- **Web Search:** Claude built-in web_search tool (5-10 searches per call)
- **Backend:** Convex (real-time DB, mutations, actions, scheduler)
- **Cost:** ~\$0.02 per vehicle enrichment (all 4 calls combined)
- **Rate Limiting:** Built-in delays (5-15s between calls) + 429 retry (60s wait)

The Structural Change

The previous single "Call 1" (base specs) has been split into Call 1A (fluids + intervals + vehicle attributes) and Call 1B (OEM part numbers). Same fetchAnthropicWithRetry, same extractJsonFromContentBlocks, same mutation patterns - but with focused prompts for each domain.

2. Stage 1: VIN Input (User Entry Points)

Two Input Paths

Users can add vehicles through two distinct paths, both converging on the same enrichment pipeline:

Path A: Manual VIN Entry

- User types 17-character VIN in app/add-vehicle.tsx
- Calls `vehicle_pipeline.decodeVin` (public action)
- User reviews decoded info, confirms via `confirmVehicleForUser`
- Background scheduler triggers `enrichVehicleSpecs`

Path B: Smartcar OBD/Connected Car

- User authenticates with Smartcar OAuth in `add-vehicle-review.tsx`
- Calls `convex/smartcar.ts:exchangeCodeAndConnect`
- VIN extracted from Smartcar API automatically
- Same `enrichVehicleSpecs` scheduled in background

Convergence Point

```
// Both paths end here:
await ctx.scheduler.runAfter(0, internal.vehicle_pipeline.enrichVehicleSpecs, {
  engineId, make, model, year, trim, engineCode, displacement, cylinders, fuelType
});
```

The `scheduler.runAfter(0, ...)` call means enrichment begins immediately but runs asynchronously - the user does not wait for AI results. The re-enrichment guard prevents duplicate work if the pipeline has already run for this engine.

3. Stage 2: NHTSA Decode + AI Normalization

NHTSA VIN Decode API

The pipeline calls the NHTSA DecodeVinValuesExtended endpoint to extract raw vehicle data. Accepts any ErrorCode that includes "0" (clean or partial decode).

Fields Extracted from NHTSA:

- make, model, year, trim, trim2, series, series2
- engineCode, cylinders, displacement, fuelType, turbo
- bodyStyle, transmission, driveType

AI Normalization (New)

NHTSA often mislabels model vs trim vs drivetrain. Example: BMW M550i xDrive - NHTSA returns Model="M550i", Trim="xdrive", but the correct mapping is Model="5 Series", Trim="M550i", Drivetrain="awd". The normalizeNhtsaWithClaude function corrects this.

Normalization Logic:

- Uses Claude Sonnet 4.5, temperature 0.1, max 4096 tokens, NO web search
- Corrects: model line, trim/submodel name, drivetrain type, engine code
- Maps brand-specific drivetrain labels: xDrive->awd, sDrive->rwd, 4matic->awd, quattro->awd

Database Upsert Chain

```
NHTSA raw -> AI Normalize -> upsertMake -> upsertModel -> upsertTrim  
-> upsertEngine -> upsertChassisVariant (if drivetrain known)
```

4. Stage 3: AI Enrichment Overview

Function: enrichVehicleSpecs (internalAction)

The main orchestrator that runs 4 sequential Claude calls with built-in guards, validation, and fallback mechanisms.

Re-Enrichment Guard

Before running anything, the pipeline checks if data already exists:

- getEngineSpecs - if engine_specs exist, skip Call 1A + 1B
- getServiceVehicleSpecsCount - if service_vehicle_specs exist, skip Call 2
- If both exist, pipeline exits early (idempotent)

Step 0: Engine Code Resolution

If NHTSA did not provide an engine code, the pipeline resolves it:

- Option 1: Single-engine trim - if trim has only 1 engine, use its code
- Option 2: AI inference - Claude returns OEM code (e.g. N63B44O2, K20C1) or "unknown"
- 5-second delay before proceeding to Call 1A

Call Sequence & Timing

Call	Purpose	Searches	Tokens	Delay
1A	Fluids, Intervals, Attributes	8	8,000	10s after
1B	OEM Part Numbers + Trim Specs	10	8,000	10s after
Gap	Sibling X-ref + Targeted Retry	5	4,000	10s after
2	Service Pricing (all services)	5	16,000	15s before

5. Call 1A: Fluids, Intervals & Vehicle Attributes

Purpose

Extract fluid specifications, structured maintenance intervals, and mechanical attributes for the vehicle. This data feeds the maintenance scheduling engine.

Key Innovation: Structured Intervals

Previously, intervals were stored as freetext like "7,500 miles or 12 months". Now the pipeline outputs BOTH the display string AND structured fields:

```
"oil_change_interval": "7,500 miles or 12 months",  
"oil_change_interval_miles": 7500,  
"oil_change_interval_months": 12,  
"oil_change_interval_status": "scheduled"
```

Interval statuses: "scheduled" | "not_applicable" | "conditional_severe" | "lifetime" | "inspect_only" | "data_unavailable"

Web Search Strategy (8 searches)

- "<year> <make> <model> owner's manual maintenance schedule"
- "<make> <engineCode> oil type capacity"
- "<year> <make> <model> <trim> service intervals"
- "<year> <make> <model> power steering type timing chain or belt"
- Searches 5-8: Fill remaining gaps dynamically

Vehicle Attributes Extracted

- power_steering_type (electric/hydraulic) - determines service applicability
- timing_system (chain/belt) - determines timing belt service applicability
- has_turbocharger - affects service complexity
- fuel_injection_type, transmission_type, drivetrain_type

Storage

- Writes to engine_specs table via storeEngineSpecs mutation
- Writes vehicle attributes to engine_specs via updateEngineAttributes
- Logs to ai_enrichment_logs via logEnrichment

6. Call 1B: OEM Part Numbers (Year-Pinned)

Purpose

Find exact OEM manufacturer part numbers for 10+ components, with per-field confidence scoring and year-specific verification.

Year-Pinning Rule (Critical Innovation)

CRITICAL RULE IN PROMPT:

"Parts MUST be verified for model year {year} specifically. Many OEM parts change between generations. If you cannot confirm a part number is correct for {year}, set confidence to 0.5 or below and note year-specific fitment unverified."

Per-Field Confidence Scoring

Each field returns: { "value": "...", "confidence": 0.0-1.0, "source": "..." }

- 0.9+ = Confirmed from OEM catalog for exact year
- 0.7-0.89 = Cross-referenced from reliable aftermarket source
- 0.5-0.69 = Same engine/platform, different year
- <=0.5 = Unverified for specific year (flagged)

OEM Parts Tracked (10 fields)

Category	Field	Example	Notes
Filtration	oil_filter_oem	15400-RTA-003	Honda format
Filtration	engine_air_filter_oem	17220-5AA-A00	Honda format
Filtration	cabin_air_filter_oem	80292-TBA-A11	Honda format
Brakes	front/rear_brake_pad_oem	45022-TBA-A01	Year-critical
Brakes	front/rear_brake_rotor_oem	45251-TBA-A02	Year-critical
Engine	spark_plug_oem	12290-R70-A01	With qty + gap
Drive	serpentine_belt_oem	31110-RBB-006	If applicable
Gasket	oil_drain_plug_gasket	94109-14000	Small but key

Trim Specs Also Extracted

- Tire sizes (front/rear), recommended pressure (PSI)
- Lug nut torque (ft-lbs), wiper blade sizes

7. Year-Pinning & Part Validation (Layer 3)

The Problem This Solves

AI models can confidently return the WRONG year's part number. A 2016 CR-V and 2017 CR-V may share the same engine but use different brake rotors due to a mid-generation change. Without validation, wrong parts get cached and quoted to customers.

validateOEMPart() - Two-Check System

Check 1: Part Number Format Validation

Per-make regex patterns catch obviously wrong part numbers:

```
Honda:      XX-XXXXXX-XX or XXXXXXXX-XXX
Toyota:     XXXXX-XXXXX or XXXXXXXXXX
BMW:        XX X X XXX XXX or XXXXXXXXXX
Mercedes:    XXXXXXXXX (3 letters + 8 digits)
Ford:        XXXX-XXXXXX variants
Hyundai/Kia: XXXXX-XXXXX
VW/Audi:     XXX XXX XXXX
+ 8 more OEM patterns | Length: 4-24 chars
```

Check 2: Year-Mismatch Detection (The Core Innovation)

After format passes, the system checks if this exact part number is already in the database mapped to a DIFFERENT year of the SAME model:

```
getOtherEnginesWithPartNumber(partNumber, excludeEngineId)

For each existing fitment:
  if (fitment.model_name === currentModel
    && fitment.year_start !== currentYear) {
    return { valid: false, flag: "year_mismatch" }
  }
```

When validation fails, the part number is stored as "N/A" instead. This prevents the exact bug where a 2017 part gets applied to a 2016 vehicle. Over time, as more vehicles flow through, the cross-reference database grows and catches more mismatches.

Validation Flow

```
AI returns part -> validatePartNumberFormat(partNum, make)
|-> FAIL -> store as "N/A"
|-> PASS -> getOtherEnginesWithPartNumber(partNum)
|-> year_mismatch found -> store as "N/A"
|-> PASS -> save to vehicle_specs
```

8. Gap Fill: Sibling Cross-Reference + AI Retry

When It Triggers

After Call 1B, the pipeline checks for fields with confidence < 0.70 or null values. These are the "gaps" that need filling.

Step 1: Sibling Cross-Reference

For engine-shared parts (same engine code = same parts), the pipeline looks up other vehicles in the database with the same engine_code and copies verified values.

Cross-Reference Safe Fields (7 fields):

- oil_filter_oem, oil_drain_plug_gasket_oem, engine_air_filter_oem
- spark_plug_oem, spark_plug_quantity, spark_plug_gap_mm
- serpentine_belt_oem

These fields are safe to cross-reference because they depend on the ENGINE, not the trim/year. Brake parts, cabin filters, etc. are NOT cross-referenced because they vary by body/trim.

Cross-referenced values get confidence: 0.80, source: "cross_ref_engine_{id}"

Step 2: Targeted AI Retry

For remaining gaps (max 8 fields), a focused Claude call with 5 web searches tries to fill specific missing data. Only accepts results with confidence >= 0.60.

Generation-Aware Search:

The gap fill prompt includes model generation ranges for smarter search context:

Honda CR-V:	[2012-2016], [2017-2022], [2023-2027]
Honda Accord:	[2013-2017], [2018-2022], [2023-2027]
Toyota Camry:	[2012-2017], [2018-2024], [2025-2027]
BMW 3 Series:	[2012-2018], [2019-2027]
BMW 5 Series:	[2011-2016], [2017-2023], [2024-2027]
Other models:	year-2 to year+2 (default range)

9. Call 2: Service Pricing Intelligence

Purpose

Generate per-service pricing estimates for THIS specific vehicle, using known specs, OEM part numbers for MSRP lookups, and web research. This powers the booking price quotes.

Context Fed to AI

- Vehicle description (year, make, model, trim, engine)
- Known oil viscosity + capacity from Call 1A
- Vehicle attributes (power steering type, timing system, turbo, drivetrain)
- All verified OEM part numbers from Call 1B (for MSRP web search)
- Full service catalog from listAllServices (slug, name, default_labor_hours)

Service Applicability Rules

The prompt automatically marks services as N/A based on vehicle attributes:

- Power steering flush -> electric PS -> is_applicable: false
- Differential service -> FWD vehicle -> is_applicable: false
- Timing belt replacement -> timing chain -> is_applicable: false

Source Rules

- **ALLOWED:** Dealer parts sites, OEM catalogs, major retailers, reputable shop menus
- **BLOCKED:** RepairPal, forums as primary source

Fallback Ladder

When exact trim data is unavailable:

- Level 1: Exact vehicle/trim match
- Level 2: Same generation/platform
- Level 3: Same engine code in closest model
- Level 4: Generic luxury/performance estimate (widest ranges)

Per-Service Output

```
{
  "slug": "oil-change",
  "is_applicable": true,
  "labor_hours": 0.5,
  "parts_cost_low": 35,
  "parts_cost_high": 55,
  "confidence_score": 0.85,
  "parts_list": [{ "item": "oil filter", "qty": 1, "price_low": 8, "price_high": 12 }],
  "tech_notes": "Uses 0W-20 full synthetic, 4.4 qts",
  "sources": ["hondapartsnow.com", "autozone.com"]
}
```

10. Data Schema & Storage Architecture

Vehicle Intelligence Tables

Table	Written By	Key Fields	Purpose
makes	NHTSA Stage	name	Brand catalog
models	NHTSA Stage	make_id, name	Model catalog
trims	NHTSA Stage	model_id, name, year	Trim catalog
engines	NHTSA Stage	trim_id, engine_code	Engine catalog
chassis_variants	AI Normal.	drivetrain, confidence	AWD/FWD/RWD
engine_specs	Call 1A	fluids, intervals, attrs	Maintenance data
vehicle_specs	Call 1B	10 OEM part columns	Part numbers
trim_specs	Call 1B	tires, wipers, lug torque	Trim-specific
service_vehicle_specs	Call 2	labor, parts cost, conf.	Per-service pricing
ai_enrichment_logs	All Calls	confidence, review_status	Audit trail
manual_review_queue	Pipeline	low-confidence items	Human review
oem_parts	Fitments	oem_part_number (unique)	Part catalog
engine_part_fitments	Fitments	engine->part, confidence	Fitment map

Data Flow

```
VIN -> NHTSA -> makes/models/trims/engines/chassis_variants
-> Call 1A -> engine_specs (fluids + intervals + attributes)
-> Call 1B -> vehicle_specs (OEM parts) + trim_specs
-> Validate -> N/A for bad parts | save verified parts
-> Gap Fill -> update vehicle_specs with siblings/AI
-> Call 2 -> service_vehicle_specs (per-service pricing)
-> Log -> ai_enrichment_logs (audit trail)
```

11. Confidence Scoring & Review System

Multi-Level Confidence

Confidence scores exist at three levels: per-field (Call 1B), per-call (Call 1A), and per-service (Call 2). This enables granular trust decisions.

Call 1B - Per-Field Confidence:

- 0.90+ = OEM catalog confirmed for exact year
- 0.70-0.89 = Cross-referenced from reliable source
- 0.50-0.69 = Same engine/platform (year-specific unverified)
- <0.50 = Unverified - auto-flagged for review

Call 2 - Service Pricing Confidence:

- 0.90+ = Both labor AND parts supported by web sources
- 0.70-0.89 = One of labor/parts inferred via fallback ladder
- <0.70 = Mostly estimated (must explain in tech_notes)

Auto-Approval Routing

The logServiceEnrichment mutation routes items automatically:

```
if (confidence >= 0.70) -> review_status: "approved"   (auto-approved)
if (confidence < 0.70) -> review_status: "pending"     (needs human review)
```

Self-Improving Cache

The system improves over time through three mechanisms:

- Sibling cross-reference: Parts verified for one engine benefit all vehicles with same engine
- Year-mismatch detection: Each new vehicle adds to the cross-reference database
- Shop confirmations: When a real job completes, the shop confirms the correct part, growing the verified cache

12. Five Key Improvements Over Previous Code

1. Structured Intervals (Call 1A)

Before: "7,500 miles or 12 months" (string that needs runtime parsing). After: oil_change_interval_miles: 7500 + oil_change_interval_months: 12 + oil_change_interval_status: "scheduled". Eliminates all string parsing.

2. Vehicle Attributes Drive Applicability (Call 1A -> Call 2)

Before: Every service shown to every vehicle. After: Vehicle attributes (power_steering_type, timing_system) automatically mark non-applicable services. Electric PS vehicles skip PS flush. Chain engines skip timing belt replacement.

3. Year-Pinned Part Numbers (Call 1B)

Before: AI returned part numbers without year verification. After: Aggressive year-pinning in the prompt forces AI to self-flag uncertainty. Parts unverified for the specific year get confidence ≤ 0.5 instead of confident wrong answers.

4. Part Validation Layer (Post-Call 1B)

Before: AI output was trusted directly. After: Two-check validation catches format errors (per-make regex) and year mismatches (cross-reference against existing database). Bad parts stored as "N/A" instead of wrong numbers.

5. Sibling Cross-Reference (Gap Fill)

Before: Missing data stayed missing. After: Engine-shared parts (oil filter, spark plugs, etc.) are automatically inherited from sibling vehicles with the same engine code. 7 "safe" fields can be cross-referenced at 0.80 confidence.

13. Practical Flow Summary

Complete Flow: User Books Service for 2016 CR-V

1. Cache Check	Check engine_part_fitments for verified parts. CACHE HIT = done, skip enrichment.
2. Cache Miss	Trigger AI enrichment with year-pinned prompt for 2016 specifically.
3. AI Returns Parts	Claude searches web, returns parts with per-field confidence scores.
4. Format Validation	Check Honda part format (XX-XXXXXX-XX). Reject malformed numbers.
5. Year-Mismatch Check	Cross-reference against database. If part already mapped to 2017 CR-V, reject.
6. Pass Validation	Save to oem_parts + vehicle_specs as unverified with confidence score.
7. Fail Validation	Flag for human review, use fallback pricing from vehicle class estimates.
8. Human Reviews	Admin reviews flagged parts in manual_review_queue. Verified = trusted cache.
9. Shop Completes Job	Mechanic confirms correct part was used. Confidence grows organically.
10. Next 2016 CR-V	Cache hit! Verified parts served instantly, no AI call needed.

The Flywheel Effect

Each vehicle that flows through the pipeline makes the system smarter. Verified parts become cache hits for identical vehicles. Sibling cross-references fill gaps for related vehicles. Year-mismatch detection catches more errors as the database grows. This means the AI enrichment cost per vehicle approaches \$0 over time as the cache fills.

14. Implementation Status & Next Steps

What Is Done (Shipped)

- Full pipeline wired: VIN path + Smartcar path both trigger AI enrichment
- All 4 Claude calls implemented (1A fluids, 1B parts, gap fill, Call 2 pricing)
- Year-pinning rule in AI prompts
- validateOEMPart with format validation (13 OEM patterns) + year-mismatch detection
- Sibling cross-reference for 7 engine-shared fields
- Re-enrichment guard (idempotent, won't re-run)
- ai_enrichment_logs, manual_review_queue tables exist in schema
- Confidence-based auto-approval routing (≥ 0.70 = approved)
- Structured intervals with status enums
- Vehicle attributes driving service applicability
- NHTSA AI normalization for model/trim/drivetrain correction

What Is Half-Done

- Seed/demo data for vehicle intelligence tables (needed to unblock UI testing)
- Manual add-car-info path does not call Convex for specs yet

Future Enhancements (Post-MVP)

- PartsTech / PartsLogic API integration for real-time dealer pricing
- Show getFullVehicleSpecPack(vin) and confidence scores in the app UI
- OEM supersession number tracking (part number evolution)
- Full analytics/funnel tracking for enrichment success rates
- Subscription tiers with different enrichment depths

Appendix: Key Functions Reference

Function	File	Purpose
enrichVehicleSpecs	vehicle_pipeline.ts	Top-level AI enrichment orchestrator
processVin	vehicle_pipeline.ts	NHTSA VIN decode + catalog upserts
normalizeNhtsaWithClaude	vehicle_pipeline.ts	Claude normalization of model/trim/drivetrain
inferEngineCodeFromVehicle	vehicle_pipeline.ts	Claude inference of OEM engine code
buildFluidsPrompt	vehicle_pipeline.ts	Constructs Call 1A prompt
buildPartsPrompt	vehicle_pipeline.ts	Constructs Call 1B prompt (year-pinned)
buildGapFillPrompt	vehicle_pipeline.ts	Constructs gap fill prompt
validateOEMPart	vehicle_pipeline.ts	Year-mismatch + format validation
validatePartNumberFormat	vehicle_pipeline.ts	Per-make regex format check
crossReferenceFromSiblings	vehicle_pipeline.ts	Sibling engine cross-reference
flattenPerFieldSpecs	vehicle_pipeline.ts	Extract flat values + confidence scores
extractJsonFromContentBlocks	vehicle_pipeline.ts	Parse JSON from Claude response
fetchAnthropicWithRetry	vehicle_pipeline.ts	HTTP call with 429 retry (60s, 2x)
getGenerationRange	vehicle_pipeline.ts	Model generation year ranges
upsertServiceVehicleSpec	vehicle_mutations.ts	Idempotent service pricing upsert
logServiceEnrichment	vehicle_mutations.ts	Audit log with auto-approve >= 0.7
getOtherEnginesWithPartNumber	vehicle_mutations.ts	Year-mismatch detection query